Tutorial 00 — Part 1
Git & GitHub
Version control for research code

Shuvam Banerji Seal

CH4114 — Molecular Simulation

14 August 2026

Course Instructor: Dr. Susmita Roy · TA: Shuvam Banerji Seal

Repository: `CH4114_Molecular_Simulation_Tutorials`

Why version control?

- Research code evolves: `analysis_final.py`, `analysis_final_v2.py`, `analysis_FINAL_v3.py`...
- Without history, a deleted good version is **gone forever**.
- Simulations are **reproducible** only if the code is versioned and taggable.

The promise of Git

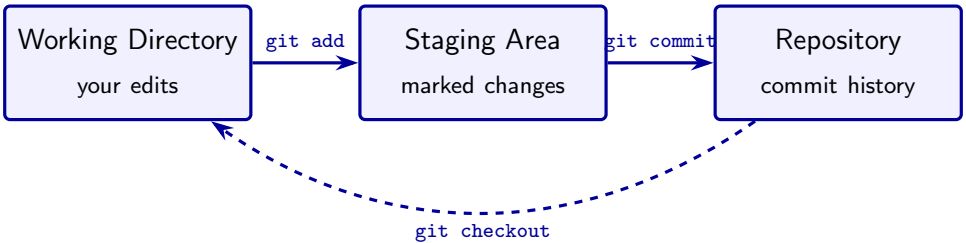
Complete history of every change, instant undo, safe experiments on branches, clean multi-person collaboration, and a snapshot you can cite.

What Git gives you

Superpower	Why it matters for simulation work
History	See exactly what changed, when, and by whom
Undo	Revert to any past state of your code
Branches	Try a new idea without breaking working code
Collaboration	Merge work from multiple people cleanly
Reproducibility	Tag a commit = a snapshot you can share

- Git takes **snapshots** (commits) of your project.
- Each commit is cheap, local, and complete.

The mental model: three areas



- **Working directory** — files you are editing.
- **Staging area** — changes marked for the next snapshot.
- **Repository** — all committed snapshots (the history).

The core workflow

The loop (memorize it)

```
git init           # once
git status         # what changed?
git add <file>     # stage changes
git commit -m "msg"
```

- Commit after every **meaningful** chunk of work.
- Run `git status` constantly.
- Write commit messages that explain *why*.

Rule

Working LJ potential script = commit. Every keystroke = no.

First-time config and history

One-time setup (per machine)

```
git config --global user.name  "You"
git config --global user.email "you@x.com"
git config --global init.defaultBranch main
```

Inspecting history

```
git log --oneline
git log --oneline --graph
git status
git diff
git diff --staged
```

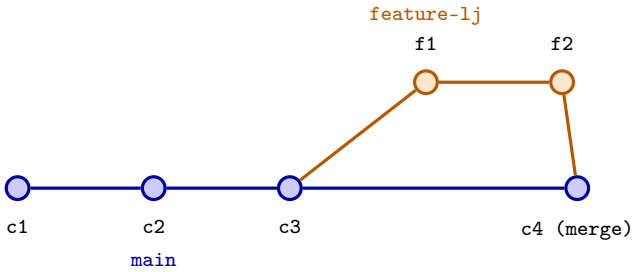
Undoing changes: three levels

Situation	Command	Effect
Edited, not staged	`git restore <file>`	discard edits
Staged by mistake	`git restore -staged <file>`	unstage, keep edits
Wrong commit message	`git commit -amend -m "msg"`	rewrite last commit

Safety

Git never throws work away silently — `git restore` only touches what you point it at. When in doubt, commit first, clean up later.

Branching and merging



- `main` stays clean; experiments live on short-lived branches.
- `git merge feature-lj` folds the branch back in.
- Golden rule: never develop directly on `main`.

Merge conflicts

Happen when two branches edit the same lines. Git marks them:

conflict markers in the file

```
<<<<<<< HEAD
E = 4 * eps * ((sigma/r)**12 - (sigma/r)**6)
=======
E = eps * ((sigma/r)**12 - (sigma/r)**6)
>>>>>>> feature-lj
```

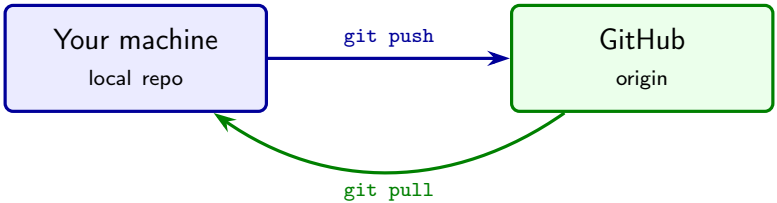
1. Edit the file by hand — keep the correct version.
2. `git add <file>`
3. `git commit -m "resolve merge conflict"`

GitHub: hosting, sharing, collaborating

Element	Purpose
Code	browse files and commit history
Issues	bug reports, feature requests, task tracking
Pull requests	propose and review changes before merging
Actions	CI/CD: run tests automatically on every push
README	the front page of your project

- GitHub is a **hosting service** for Git repositories.
- Cloud backup + collaboration hub + public portfolio.

Local repository ↔ GitHub



- `git remote add origin <url>` — connect once.
- `git push -u origin main` — first push; afterwards plain `git push`.
- `git clone <url>` — copy a remote repo to your machine.

Collaboration loop: issues, branches, pull requests

1. Open an **issue** describing the task.
2. Create a branch: `git checkout -b fix-lj-preprint`
3. Commit and push: `git push -u origin fix-lj-preprint`
4. Open a **pull request** on GitHub: "Closes #12 — fixes LJ prefactor".
5. Reviewer comments; push more commits; **merge**.
6. Pull locally: `git pull`

Why PRs?

Nothing lands on `main` unreviewed — history records intent.

The gh CLI: GitHub from your terminal

Authenticate once

```
gh auth login
gh auth status
```

Daily commands

```
gh repo create my-repo --public --source . --push
gh issue create --title "Add MC code" --body "..."
gh pr create --title "..." --body "..."
gh repo view --web
```

- `gh repo create -source .  -push` is exactly how this repository was published.

Cheat sheet — print this

```
# setup
git config --global user.name "N"
git config --global user.email "e"

# everyday
git status
git diff
git log --oneline
git add <file> | git add .
git commit -m "msg"
git push | git pull

# branches
git branch <name>
git checkout -b <name>
git switch <name>
git merge <name>
git branch -d <name>
```

```
# undo
git restore <file>
git restore --staged <file>
git commit --amend -m "msg"

# remote
git remote -v
git remote add origin <url>
git clone <url>

# github
gh auth login
gh repo create NAME --public \
   --source . --push
gh pr create
gh issue create
```

Check yourself

1. What is the difference between `git add` and `git commit`?
2. A collaborator changed `main` while you worked on a branch. Which two commands bring their changes in?
3. You committed with a typo'd message. Which one command fixes it?
4. Which command creates a GitHub repository from the current folder and pushes it?

Next up

Part 2: organizing your project like a professional. Part 3: uv environments.